ADR-0001 : Persistance en SQLite « document-store » avec cache write-through

- **Statut :** Accepté
- **Date :** 2026-07-21
- **Décideurs :** Driss OUAAZIZ — Software Architect

Contexte et problème

FileFlow doit persister l'état de configuration de plusieurs domaines : utilisateurs, groupes, liens de partage, dossiers collaboratifs, partages internes, demandes d'upload et favoris. La première implémentation conservait chaque collection dans un fichier JSON dédié (`.fileflow_*.json`), rechargé puis **réécrit intégralement à chaque mutation**.

Cette approche présente deux limites majeures :

- **Coût d'écriture en O(n)** : modifier une seule entité impose de resérialiser et de réécrire l'intégralité de la collection, ce qui dégrade les performances et multiplie les risques de corruption en cas d'interruption pendant l'écriture.
- **Absence de garanties transactionnelles** : deux mutations concurrentes peuvent se perdre mutuellement (lecture-modification-écriture non atomique).

Le produit étant distribué sous forme de **binaire unique auto-hébergé**, toute solution retenue doit rester embarquée, sans service externe à administrer.

Facteurs de décision

- Écritures unitaires peu coûteuses (mutation d'une entité = une seule ligne).
- Durabilité et intégrité des données face aux arrêts brutaux.
- Absence de dépendance à un service externe (contrainte d'auto-hébergement).
- Rétrocompatibilité avec les données existantes au format JSON.
- Simplicité de maintenance et faible surface d'exploitation.

Options envisagées

1. **Fichiers JSON réécrits en totalité (existant)** — Simple, mais écritures en O(n), pas d'atomicité, corruption possible. Ne passe pas à l'échelle.
2. **SQLite en modèle relationnel normalisé** — Robuste, mais impose un schéma et des migrations par domaine, pour un gain limité : les entités sont lues et écrites en bloc, sans requêtes relationnelles transverses.
3. **Base externe (PostgreSQL, etc.)** — Écarté : contredit la contrainte de binaire auto-hébergé et alourdit l'exploitation (service, sauvegardes, supervision supplémentaires).
4. **SQLite en tables « document »** *(retenu)* — Une table par domaine, de forme `(key TEXT PRIMARY KEY, data TEXT)` où `data` contient le JSON de l'entité. Conserve un format de sérialisation identique à l'existant tout en apportant des écritures ligne à ligne et les garanties de SQLite.

Décision

Adopter **SQLite comme magasin de documents**. Chaque domaine correspond à une table `(key, data)` (voir `TABLES` : `users`, `groups`, `shares`, `collab`, `internal_shares`, `upload_requests`, `favorites`). Chaque store conserve en mémoire une `HashMap` servant de cache de lecture et **écrit en traversée (write-through) une ligne à la fois** à chaque mutation (`put` / `delete`), supprimant les réécritures globales.

Caractéristiques :

- **Mode WAL** (`journal_mode = WAL`, `synchronous = NORMAL`) : lecteurs concurrents et écrivain unique, commits rapides et durables.
- **Connexion unique** partagée via `Arc<tokio::sync::Mutex<Connection>>` : sérialise les écritures, ce qui suffit à l'échelle visée.
- **Migration automatique** : au premier démarrage, si une table est vide et qu'un ancien fichier `.fileflow_*.json` existe, son contenu est importé ligne à ligne dans SQLite, puis le fichier hérité est ignoré.
- Le nom de table provient exclusivement de la constante `TABLES` (jamais d'une entrée utilisateur), ce qui rend son interpolation dans le SQL sûre ; toutes les valeurs passent par des requêtes paramétrées.

Le **journal d'audit reste distinct**, en fichier append-only (JSONL) : il s'agit déjà d'ajouts en O(1), sans problème de réécriture (voir sa propre politique de rotation).

Conséquences

Positives

- Écriture d'une mutation ramenée à un `INSERT … ON CONFLICT DO UPDATE` sur une seule ligne (coût constant).
- Durabilité et atomicité des écritures apportées par SQLite/WAL.
- Aucun service externe : le produit reste un binaire unique auto-hébergé.
- Transition transparente pour les déploiements existants (import automatique).

Négatives

- La connexion unique sous mutex **sérialise toutes les écritures** ; le débit d'écriture est borné par un seul écrivain.
- Le cache mémoire duplique l'état persistant ; toute mutation doit impérativement écrire à la fois le cache et la base.

Risques résiduels

- Sous très forte concurrence d'écriture, le mutex unique peut devenir un point de contention. **Piste d'évolution** : introduire un pool de connexions (par ex. `r2d2`) pour paralléliser les écritures si le besoin se confirme.

Références

- Documentation SQLite — *Write-Ahead Logging* : https://www.sqlite.org/wal.html.
- ADR-0007 (uploads résumables) pour l'état conservé hors de ce magasin.
- Code : `backend/src/db.rs`, stores `backend/src/{users,shares,collab,internal_shares,upload_requests,favorites}.rs`.